第二章 线性表 · 实践部分：顺序表与单链表的实现

> 数据结构教学案例整理 | 案例：学生成绩表的顺序存储、教师名单的链式存储

案例一：学生成绩管理模块设计（顺序表）

1. 案例描述

设计一个学生成绩管理模块，用**顺序表**存储一个班级的学生成绩。要求：

- 初始化一个空的成绩表（顺序表）。
- 依次输入 N 名学生的学号和成绩，存入顺序表。
- 支持在第 i 个位置**插入**一名学生的成绩。
- 支持**删除**第 i 个位置上的学生成绩，并把被删的记录带回来。
- 支持**按学号查找**并输出该学生的成绩。
- 每完成一次操作，都要**遍历输出**当前成绩表。

2. 案例分析（数据结构设计）

- **数据元素：** 一名学生的成绩信息。
- **数据项：** 学号（长整型 `long id`）、成绩（整型 `int score`）。
- **逻辑结构：** 学生成绩表是一条线性表——按录入顺序一个接一个排列，除第一个和最后一个外，每名学生都有“前一名”和“后一名”。
- **存储结构：** 采用**顺序存储**，用结构体定义顺序表类型 `SqList`：
  - `data[MAXSIZE]` 用数组顺序存放成绩记录；
  - `length` 记录当前学生人数。
- **算法设计：**
  - 插入：先判位置合法、判表满，再从**后往前**依次后移，最后填空、`length++`；
  - 删除：先判位置合法，保存被删元素，再从**前往后**依次前移，最后 `length--`；
  - 查找：按学号逐个比较（线性查找），找到返回位序。

3. 核心C语言代码清单

```c
#include <stdio.h>

#define MAXSIZE 100         // 顺序表的最大容量

// 1. 定义数据结构
typedef struct {
    long id;                // 学号
    int  score;             // 成绩
} ElemType;                 // 数据元素：一名学生的成绩信息
```

```c
typedef struct {
    ElemType data[MAXSIZE]; // 数组定义，用来顺序存储多个学生成绩
    int      length;        // 当前学生人数
} SqList;

// 2. 初始化：把顺序表置为空表
void InitList(SqList *L) {
    L->length = 0;
}

// 3. 插入：在顺序表 L 的第 i 个位置插入元素 e
int ListInsert(SqList *L, int i, ElemType e) {
    int j;
    if (i < 1 || i > L->length + 1) {      // 插入位置不合法
        return 0;
    }
    if (L->length >= MAXSIZE) {            // 表已满
        return 0;
    }
    for (j = L->length - 1; j >= i - 1; j--) {
        L->data[j + 1] = L->data[j];       // 从后往前逐个后移，腾出空位
    }
    L->data[i - 1] = e;                    // 空出的位置放入新元素
    L->length++;                           // 表长加 1
    return 1;
}

// 4. 删除：删除第 i 个元素，用 *e 带回，成功返回 1
int ListDelete(SqList *L, int i, ElemType *e) {
    int j;
    if (i < 1 || i > L->length) {          // 删除位置不合法
        return 0;
    }
    *e = L->data[i - 1];                   // 先把被删元素保存起来
    for (j = i; j < L->length; j++) {
        L->data[j - 1] = L->data[j];       // 从前往后逐个前移，填上坑
    }
    L->length--;                           // 表长减 1
    return 1;
}

// 5. 按学号查找：返回位序（找不到返回 0）
int LocateElem(SqList L, long id) {
    int i;
    for (i = 0; i < L.length; i++) {
        if (L.data[i].id == id) {          // 逐个比对数据项
            return i + 1;
        }
    }
    return 0;
}

// 6. 遍历输出
```

```c
void PrintList(SqList L) {
    int i;
    printf("学号\t成绩\n");
    for (i = 0; i < L.length; i++) {
        printf("%ld\t%d\n", L.data[i].id, L.data[i].score);
    }
}

int main() {
    SqList L;
    ElemType e;
    int n, i, pos;
    long id;

    InitList(&L);                           // 初始化空表

    printf("请输入学生人数: ");
    scanf("%d", &n);

    printf("请依次输入学号和成绩(如 1001,92):\n");
    for (i = 0; i < n; i++) {
        scanf("%ld,%d", &e.id, &e.score);
        ListInsert(&L, L.length + 1, e);    // 依次插到表尾
    }
    printf("--- 初始成绩表 ---\n");
    PrintList(L);

    // 演示插入：在第 2 个位置插入一名学生
    e.id = 1009;
    e.score = 88;
    if (ListInsert(&L, 2, e)) {
        printf("--- 插入成功，当前成绩表 ---\n");
        PrintList(L);
    }

    // 演示删除：删除第 3 个位置上的学生
    if (ListDelete(&L, 3, &e)) {
        printf("已删除学生：%ld，成绩：%d\n", e.id, e.score);
        printf("--- 删除后成绩表 ---\n");
        PrintList(L);
    }

    // 演示查找
    printf("请输入要查找的学号: ");
    scanf("%ld", &id);
    pos = LocateElem(L, id);
    if (pos > 0) {
        printf("找到该学生，位于第 %d 个位置，成绩 %d\n", pos, L.data[pos - 1].score);
    } else {
        printf("查无此人! \n");
    }
    return 0;
}
```

案例二：教学质量评价系统教师名单管理（单链表 · 技能提高）

1. 案例描述

设计一个教师名单管理模块，用**单链表**动态存储教师信息（表长不固定，支持随时增删）。要求：

- 初始化一个带头结点的空链表。
- 用**尾插法**依次输入 N 名教师（工号、姓名、权限），保持输入顺序。
- 支持在第 i 个位置**插入**一名新教师。
- 支持**删除**第 i 个结点上的教师信息，并输出被删教师。
- 支持**遍历**输出全部教师，并根据权限值显示身份（1 → 教学督导，其他 → 普通教师）。

2. 案例分析（数据结构设计）

- **数据元素：** 一位教师的信息。
- **数据项：** 工号（长整型 `long id`）、姓名（字符数组 `char name[20]`）、权限（整型 `int pms`）、指向下一个结点的指针（`struct LNode *next`）。
- **逻辑结构：** 教师名单同样是**线性表**，但表长会动态变化，适合用链式存储。
- **存储结构：** 采用**链式存储**，用结构体定义结点类型 `LNode`，并用 `LinkList` 作为指向结点的指针类型；设置**头结点**统一插入、删除的处理。
- **算法设计：**
  - 建表：**尾插法**（`r` 指针始终指向表尾），得到与输入顺序一致的表；
  - 插入：先 `GetElem(L, i-1)` 找到前驱，再按**先接后、再断前**的顺序改指针；
  - 删除：先找前驱 p，令 `p->next = q->next` 跳过被删结点，再 `free(q)` 释放内存；
  - 查找：从头结点的下一个结点开始，顺着 `next` 逐个推进。

3. 核心C语言代码清单

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 1. 定义数据结构：结点
typedef struct LNode {
    long id;                // 工号
    char name[20];          // 姓名
    int  pms;               // 权限：1 = 教学督导，2 = 普通教师
    struct LNode *next;     // 指针域：指向下一个结点
} LNode, *LinkList;

// 2. 初始化：建立带头结点的空链表
int InitList(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));    // 申请头结点
    if (*L == NULL) {
        return 0;
```

```
    }
    (*L)->next = NULL;                          // 头结点的 next 置空
    return 1;
}

// 3. 尾插法建表：依次插入 n 名教师（保持输入顺序）
void CreateListTail(LinkList L, int n) {
    LNode *r = L;                               // r 始终指向表尾
    LNode *s;
    int i;
    for (i = 0; i < n; i++) {
        s = (LNode *)malloc(sizeof(LNode));// 申请新结点
        scanf("%ld,%s,%d", &s->id, s->name, &s->pms);
        r->next = s;                            // 新结点接到表尾
        r = s;                                  // 尾指针后移
    }
    r->next = NULL;                             // 表尾结点的 next 置空
}

// 4. 按位查找：返回第 i 个结点的指针（i = 0 返回头结点）
LNode *GetElem(LinkList L, int i) {
    int j = 1;
    LNode *p = L->next;                         // 从第一个真元素结点开始
    if (i < 0) {
        return NULL;
    }
    if (i == 0) {
        return L;
    }
    while (p != NULL && j < i) {                // 顺着 next 一个一个往后走
        p = p->next;
        j++;
    }
    return p;
}

// 5. 插入：在第 i 个位置插入一名新教师
int ListInsert(LinkList L, int i, LNode e) {
    LNode *p = GetElem(L, i - 1);               // 找到第 i-1 个结点（前驱）
    LNode *s;
    if (p == NULL) {                            // 位置不合法
        return 0;
    }
    s = (LNode *)malloc(sizeof(LNode));
    if (s == NULL) {
        return 0;
    }
    s->id = e.id;
    strcpy(s->name, e.name);                    // 字符串要用 strcpy 复制
    s->pms = e.pms;
    s->next = p->next;                          // ① 先接后：新结点接上后面的链
    p->next = s;                                // ② 再断前：前驱指向新结点
    return 1;
```

```c
}

// 6. 删除：删除第 i 个结点，数据用 *e 带回
int ListDelete(LinkList L, int i, LNode *e) {
    LNode *p = GetElem(L, i - 1);          // 找第 i-1 个结点（前驱）
    LNode *q;
    if (p == NULL || p->next == NULL) {    // 前驱不存在或该结点不存在
        return 0;
    }
    q = p->next;                           // q 指向要被删除的结点
    *e = *q;                               // 把数据备份出来
    p->next = q->next;                     // 跳过 q，把它从链上摘下来
    free(q);                               // 释放被删结点的内存（关键！）
    return 1;
}

// 7. 遍历输出
void PrintList(LinkList L) {
    LNode *p = L->next;                    // 从第一个真元素结点开始
    printf("工号\t姓名\t权限\n");
    while (p != NULL) {
        printf("%ld\t%s\t%s\n", p->id, p->name,
               p->pms == 1 ? "教学督导" : "普通教师");  // 权限二次判断
        p = p->next;                       // 顺着 next 往后走
    }
}

int main() {
    LinkList L;
    LNode e;
    int n;

    InitList(&L);                          // 初始化空链表

    printf("请输入教师人数: ");
    scanf("%d", &n);

    printf("请依次输入工号、姓名、权限(如 2001,张三,1):\n");
    CreateListTail(L, n);                  // 尾插法建表

    printf("--- 当前教师名单 ---\n");
    PrintList(L);

    // 演示插入：在第 2 个位置插入一名教师
    e.id = 2008;
    strcpy(e.name, "李四");
    e.pms = 2;
    if (ListInsert(L, 2, e)) {
        printf("--- 插入成功，当前教师名单 ---\n");
        PrintList(L);
    }

    // 演示删除：删除第 1 个结点
```

```
    if (ListDelete(L, 1, &e)) {
        printf("已删除教师：%ld %s\n", e.id, e.name);
        printf("--- 删除后教师名单 ---\n");
        PrintList(L);
    }
    return 0;
}
```

教学知识点总结

这两个案例很好地串联了第二章"线性表的结构分析与应用"的核心知识：

- **数据封装：** 用 `typedef struct` 把不同类型的数据（学号/成绩、工号/姓名/权限/指针）组合成一个逻辑单元，形成顺序表的元素类型和链表的结点类型。
- **两种存储结构：**
  - **顺序存储**：`SqList { data[MAXSIZE]; length; }`，一段连续空间 + 表长计数器，支持随机访问；
  - **链式存储**：`LNode { data; *next; }` + 头结点 + 头指针 `LinkList L`，靠指针把分散的结点串起来。
- **插入算法对比：**
  - 顺序表插入：判位置 / 判表满 → **从后往前**后移元素 → 填空 → `length++`（O(n)，耗时在搬数据）；
  - 单链表插入：`GetElem` 找前驱 → 申请结点 → **先接后、再断前**（只改指针，O(1)）。
- **删除算法对比：**
  - 顺序表删除：保存被删元素 → **从前往后**前移元素 → `length--`（O(n)）；
  - 单链表删除：找前驱 → `p->next = q->next` 摘链 → `free(q)` 释放内存（O(1)，千万别忘记 `free`）。
- **指针操作：** `p->next`、`p = p->next`、`*e = *q` 等指针访问贯穿全链表操作；`malloc` 申请结点、`free` 释放结点是链式存储的"呼吸动作"。
- **字符串处理：** 姓名、密码等字符串不能直接用 `=` 赋值，必须使用 `string.h` 库中的 `strcpy()`；比较要用 `strcmp()`。
- **算法逻辑：** 两种结构都用到了**线性查找（Linear Search）**——顺序表按下标扫描找学号，单链表顺着 `next` 扫描找工号；循环中找到即 `break`，用边界条件（`i == length` / `p == NULL`）判定"未找到"。
- **选型思想：** 人数固定、以查询为主 → **顺序表**（成绩表）；人数不定、增删频繁 → **单链表**（教师名单）。